

UVENT

Texnik Topshiriq (TZ)

Restoran va to'yxonalarni onlayn qidirish, filtrlash va bron qilish platformasi

Backend: Python / Django + Django REST Framework

Frontend: Mobil ilova (React Native / Flutter)

Versiya: 1.0

Jamoa: UVENT Team

Eslatma: hujjatning ba'zi joylarida (masalan restoran uchun oylik obuna narxi) aniq raqam berilmagani uchun mantiqiy taxmin kiritilgan va bu alohida belgilab qo'yilgan. Jamoa bilan kelishib, shu raqamlarni real qiymatga almashtirish kerak bo'ladi.

1. Loyiha haqida qisqacha

UVENT — foydalanuvchiga yaqin atrofdagi restoran va to'yxonalarni qidirish, filtrlash, xona/stol/zal va menyu/paketni ko'rish hamda aniq sana-vaqtga bron qilish imkonini beruvchi mobil platforma. Restoran/to'yxona egalari esa platformaga oddiy foydalanuvchi sifatida kirib, 'Restoran/To'yxona ochish' arizasi orqali biznes profiliga o'tadi. To'lov jarayoni to'liq **Telegram orqali qo'lda (admin bilan muloqot orqali)** amalga oshiriladi — platforma ichida avtomatik to'lov integratsiyasi ushbu versiyada yo'q.

2. Rollar tizimi

Platformada rasman **2 ta rol** mavjud:

Rol	Tavsif
user (oddiy foydalanuvchi)	Standart holat. Ro'yxatdan o'tgan har bir kishi shu rolda boshlaydi. Restoran/to'yxona qidiradi, ko'radi, bron qiladi, sharh qoldiradi.
business (biznes admin)	'Restoran/To'yxona ochish' arizasi qabul qilingandan so'ng shu rolga o'tadi. O'z biznes-profilini, xona/zal, menyu/paketlarni boshqaradi, kelgan bronlarni ko'radi.

Super-admin — bu alohida rol emas, balki Django'ning `is_staff` / `is_superuser` maydoni orqali beriladigan platforma egasi huquqi. U arizalarni ko'rib chiqadi, to'lovni tasdiqlaydi va rolni businessga o'zgartiradi, umumiy statistikani ko'radi. Super-admin soni kam (odatda 1-3 kishi) va Django admin panel orqali ishlaydi.

3. Ro'yxatdan o'tish va autentifikatsiya

3.1. Ro'yxatdan o'tish maydonlari

Maydon	Turi	Talab
full_name	matn	To'liq ism-familiya, majburiy
email	email	Majburiy, unique
phone_number	matn	Majburiy, unique, SMS-kod orqali tasdiqlanadi
username	matn	Majburiy, unique, pastdagi qoidalarga mos bo'lishi shart
password	matn	Majburiy, minimal 8 belgi

3.2. Username qoidalari (MUHIM)

username faqat quyidagi belgilardan iborat bo'lishi mumkin:

- Kichik lotin harflari: a-z
- Raqamlar: 0-9
- Pastki chiziqcha: _

Bosh harf, probel, defis (-), va boshqa maxsus belgilar (@, ., ! va h.k.) ishlatilishi TAQIQLANADI.

To'g'ri misollar	Noto'g'ri misollar	Sabab
shohona	Shohona	bosh harf bor
dilmurod_ota	dilmurod-ota	defis bor
ozod777	dilmurod ota	probel bor
uvent_toshkent	shoh.ona	nuqta bor

Django validatori

```
# accounts/validators.py
import re
from django.core.exceptions import ValidationError

def validate_username(value):
    pattern = r'^[a-z0-9_]{3,30}$'
    if not re.match(pattern, value):
        raise ValidationError(
            "Username faqat kichik lotin harflari, raqamlar va pastki "
            "chiziqlar ( _ ) dan iborat bo'lishi kerak."
        )
```

Uzunlik chegarasi taxmin sifatida 3–30 belgi qilib qo'yildi — jamoa xohlasa o'zgartirishi mumkin.

3.3. Autentifikatsiya oqimi

- Foydalanuvchi full_name, email, phone_number, username, password kiritadi.
- Tizim phone_number'ga SMS-kod yuboradi (Eskiz.uz yoki Play Mobile kabi mahalliy SMS-shlyuz orqali).
- Kod tasdiqlangach User obyekti yaratiladi, role = 'user' bilan.
- Kirish uchun JWT token (access + refresh) beriladi (django-rest-framework-simplejwt).

4. “Restoran/To'yxona ochish” — biznes ariza oqimi

Bu — loyihaning eng muhim biznes-jarayoni. To'liq bosqichlar:

4.1. Bosqichlar

1-qadam — Ariza yuborish

Oddiy foydalanuvchi profil bo'limida 'Restoran yoki To'yxona ochish' tugmasini bosadi. Forma avtomatik ravishda ro'yxatdan o'tishda kiritilgan full_name, username, phone_number, email ma'lumotlarini oldindan to'ldiradi (foydalanuvchi qayta kiritmaydi). Qo'shimcha faqat 2 ta maydon so'raladi: biznes turi (Restoran yoki To'yxona) va biznes nomi (masalan 'Shohona to'yxonasi').

2-qadam — Tizim javobi (ekran matni)

Foydalanuvchi 'Ariza yuborish' tugmasini bosgach, ekranda quyidagi xabar chiqadi:

■ Arizangiz qabul qilindi! Sizga 7 kunlik BEPUL Pro versiya ochib berildi — shu muddat ichida platformaning barcha imkoniyatlaridan foydalanishingiz mumkin. Obunani davom ettirish uchun Telegram orqali administrator bilan bog'laning: @uvent_admin

3-qadam — Avtomatik 7 kunlik trial

Ariza yuborilgan zahoti, tizim:

- User.role'ni 'user'dan 'business'ga avtomatik o'zgartiradi (marketing maqsadida — biznes egasi platformani sinab ko'rishi va 'yoqib qolishi' uchun to'lovdan oldin kirish beriladi),
- Business obyektini yaratadi, subscription_status = 'trial', trial_ends_at = bugun + 7 kun,
- BusinessApplication yozuvini status='pending_payment' bilan saqlaydi.

Shu 7 kun davomida biznes egasi to'liq huquqda — xona/zal qo'shishi, menyu/paket kiritishi, bronlarni ko'rishi mumkin. Tavsiya: trial davrida ham profil ommaviy qidiruvda ko'rinsin, chunki bu ularga real bron kelishini ko'rsatib, to'lovga undaydi.

4-qadam — Telegram orqali muloqot va to'lov

Foydalanuvchi Telegram'da admin bilan (yoki maxsus Telegram bot orqali) bog'lanadi, to'lov shartlarini biladi va to'lovni tashqi to'lov usuli orqali (Payme/Click havolasi yoki karta-kartaga) amalga oshiradi. Bu qism platforma ichida emas, balki Telegram chatida bo'ladi.

5-qadam — Admin tasdig'i

Super-admin to'lovni ko'rgach, Django admin panelida tegishli BusinessApplication yozuviga kirib, 'To'lovni tasdiqlash' tugmasini bosadi. Bu amal natijasida:

- BusinessApplication.status = 'approved'
- Business.subscription_status = 'active'
- Business.subscription_ends_at = bugun + 30 kun
- is_paid = True

6-qadam — Muddat tugashi

Agar 7 kunlik trial tugagunga qadar to'lov tasdiqlanmasa: Business.subscription_status = 'expired' bo'ladi (Celery orqali avtomatik tekshiruv), biznes profili ommaviy qidiruvda yashiriladi va dashboard'da 'Obunangiz tugadi, davom ettirish uchun @uvent_admin bilan bog'laning' degan bloklash ekrani chiqadi. Role 'business' bo'lib qoladi (foydalanuvchi istalgan vaqt to'lov qilib qayta faollashtirishi mumkin), faqat funksionallik cheklanadi.

4.2. Obuna narxlari

Biznes turi	Oylik narx	Izoh
To'yxona	300 000 so'm / oy	Foydalanuvchi bergan aniq raqam
Restoran	(taxminiy) 150 000–200 000 so'm / oy	Aniqlashtirish kerak — jamoa bilan kelishilishi lozim

4.3. Holatlar diagrammasi (status matni)

```
graph TD
    user(ariza yuboradi) --> v1(( ))
    v1 --> business(business + subscription_status = TRIAL (7 kun, avtomatik))
    business --> v2(( ))
    v2 --> active(to'lov qilindi + admin tasdiqladi --> subscription_status = ACTIVE (30 kun))
    active --> v3(( ))
    v3 --> expired1(muddat tugaydi --> yangilanmasa --> EXPIRED)
    expired1 --> expired2(to'lov qilinmadi, 7 kun tugadi --> subscription_status = EXPIRED (profil yashirin))
```

5. Telegram integratsiyasi

To'lov o'zi Telegram orqali qo'lda amalga oshsa-da, jarayonni tezlashtirish uchun tizimga bildirishnoma uchun Telegram bot qo'shish tavsiya etiladi (python-telegram-bot yoki oddiy Telegram Bot API orqali):

- Yangi BusinessApplication yaratilganda, bot super-adminning Telegram guruhiga avtomatik xabar yuboradi: foydalanuvchi ismi, telefon raqami, username, biznes turi va nomi bilan.
- Bu — to'lovni avtomatlashtirish emas, faqat adminni tezroq xabardor qilish uchun.
- Foydalanuvchiga ham bron tasdig'i, eslatmalar shu bot orqali yuborilishi mumkin (ixtiyoriy, keyingi bosqichda).

6. Qidiruv, filtrlash va dashboard talablari

6.1. Bosh sahifa (Dashboard)

Mobil ilovaning bosh sahifasida:

- Yuqorida qidiruv qatori — nomi bo'yicha umumiy qidiruv (masalan 'Shohona' deb yozsa, shu nomdagi restoran/to'yxona chiqadi).
- **Masofa/yaqinlik** filtri — foydalanuvchi geolokatsiyasidan radius bo'yicha (1 km, 3 km, 5 km, 10 km+).
- **Sana va vaqt** filtri — foydalanuvchi kerakli sana va soatni tanlaydi, faqat o'sha vaqtga bo'sh joyi bor restoran/to'yxonalar chiqadi.
- **Turi** filtri — restoran / to'yxona.
- Narx toifasi (ixtiyoriy, keyingi bosqich).
- Natijalar ro'yxati/kartochkalarida har birining nomi, surati, masofasi, reytingi va qisqa manzili ko'rsatiladi.

6.2. Qidiruv API talablari

- GET /api/businesses/?search=<nom> — nom bo'yicha umumiy qidiruv (Django ORM contains yoki PostgreSQL full-text search)
- GET /api/businesses/?lat=..&lng=..&radius_km=.. — geolokatsiya bo'yicha filtr (Haversine formula yoki PostGIS)
- GET /api/businesses/?date=..&time=.. — faqat shu vaqtga mos Availability yozuvi mavjud bo'lganlarni qaytaradi
- Barcha filtrlar birga kombinatsiyalanib ishlatilishi mumkin (masalan: nomi + masofa + vaqt)

7. Django ilovalari (apps) arxitekturası

Loyiha bir nechta mustaqil, lekin bir-biriga bog'liq Django ilovalariga bo'linadi. Umumiy (common) ilova — accounts. Qolgan **5 ta** ilova, har birida kamida 2 tadan model bor va ular FK orqali bir-biriga bog'langan.

```
uvent_backend/  
■■■ accounts/      (COMMON - foydalanuvchi, autentifikatsiya, ariza)  
■■■ businesses/    (restoran/to'yxona va uning xona/zallari)  
■■■ catalog/       (menyu va paketlar)  
■■■ reservations/  (bron va bo'sh vaqt jadvali)  
■■■ reviews/      (sharh va reyting)  
■■■ subscriptions/ (obuna va to'lov tarixi)
```

7.1. accounts (COMMON ilova)

Barcha boshqa ilovalar shu yerdagi User modeliga bog'lanadi.

```
# accounts/models.py  
from django.contrib.auth.models import AbstractUser  
from django.db import models  
from .validators import validate_username  
  
class User(AbstractUser):  
    ROLE_CHOICES = (  
        ('user', 'Oddiy foydalanuvchi'),  
        ('business', 'Biznes admin'),  
    )  
    username = models.CharField(max_length=30, unique=True,  
                                validators=[validate_username])  
    full_name = models.CharField(max_length=150)  
    email = models.EmailField(unique=True)  
    phone_number = models.CharField(max_length=13, unique=True)  
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='user')  
    is_phone_verified = models.BooleanField(default=False)  
    created_at = models.DateTimeField(auto_now_add=True)  
  
class BusinessApplication(models.Model):  
    STATUS_CHOICES = (  
        ('pending_payment', "To'lov kutilmoqda"),  
        ('approved', 'Tasdiqlangan'),  
        ('rejected', 'Rad etilgan'),  
    )  
    BUSINESS_TYPE_CHOICES = (  
        ('restaurant', 'Restoran'),  
        ('venue', "To'yxona"),  
    )  
    applicant = models.ForeignKey(User, on_delete=models.CASCADE,  
                                  related_name='applications')  
    business_type = models.CharField(max_length=15, choices=BUSINESS_TYPE_CHOICES)  
    business_name = models.CharField(max_length=200)  
    status = models.CharField(max_length=20, choices=STATUS_CHOICES,  
                              default='pending_payment')  
    submitted_at = models.DateTimeField(auto_now_add=True)  
    approved_at = models.DateTimeField(null=True, blank=True)  
    approved_by = models.ForeignKey(User, null=True, blank=True,  
                                    on_delete=models.SET_NULL,  
                                    related_name='approved_applications')
```

7.2. businesses

```
# businesses/models.py
from django.db import models
from accounts.models import User, BusinessApplication

class Business(models.Model):
    TYPE_CHOICES = (
        ('restaurant', 'Restoran'),
        ('venue', "To'yxona"),
    )
    owner = models.ForeignKey(User, on_delete=models.CASCADE,
                              related_name='businesses')
    application = models.OneToOneField(BusinessApplication,
                                       on_delete=models.CASCADE,
                                       related_name='business')
    name = models.CharField(max_length=200)
    business_type = models.CharField(max_length=15, choices=TYPE_CHOICES)
    address = models.CharField(max_length=255)
    latitude = models.FloatField()
    longitude = models.FloatField()
    description = models.TextField(blank=True)
    cover_photo = models.ImageField(upload_to='business_covers/',
                                    null=True, blank=True)
    is_visible = models.BooleanField(default=True)
    rating_avg = models.FloatField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)

class Room(models.Model):
    ROOM_TYPE_CHOICES = (
        ('vip', 'VIP xona'),
        ('standard', 'Oddiy zal'),
        ('outdoor', 'Tashqi terrasa'),
        ('hall', "Katta zal (to'yxona)"),
    )
    business = models.ForeignKey(Business, on_delete=models.CASCADE,
                                related_name='rooms')
    name = models.CharField(max_length=100)
    room_type = models.CharField(max_length=15, choices=ROOM_TYPE_CHOICES)
    capacity = models.PositiveIntegerField()
    price_per_slot = models.DecimalField(max_digits=12, decimal_places=2)
```

7.3. catalog

```
# catalog/models.py
from django.db import models
from businesses.models import Business

class MenuItem(models.Model):
    business = models.ForeignKey(Business, on_delete=models.CASCADE,
                                related_name='menu_items')
    name = models.CharField(max_length=150)
    description = models.TextField(blank=True)
    price = models.DecimalField(max_digits=12, decimal_places=2)
    photo = models.ImageField(upload_to='menu_items/', null=True, blank=True)
    is_available = models.BooleanField(default=True)

class Package(models.Model):
    """To'yxonalar uchun tayyor paket (zal + taom + dekor)."""
    business = models.ForeignKey(Business, on_delete=models.CASCADE,
                                related_name='packages')
    name = models.CharField(max_length=150)
    description = models.TextField()
    price_per_person = models.DecimalField(max_digits=12, decimal_places=2)
    min_guests = models.PositiveIntegerField(default=50)
```

7.4. reservations


```

# reservations/models.py
from django.db import models
from accounts.models import User
from businesses.models import Business, Room

class Availability(models.Model):
    business = models.ForeignKey(Business, on_delete=models.CASCADE,
                                related_name='availabilities')
    room = models.ForeignKey(Room, on_delete=models.CASCADE,
                             related_name='availabilities')
    date = models.DateField()
    start_time = models.TimeField()
    end_time = models.TimeField()
    is_booked = models.BooleanField(default=False)

    class Meta:
        unique_together = ('room', 'date', 'start_time')

class Reservation(models.Model):
    STATUS_CHOICES = (
        ('pending', 'Kutilmoqda'),
        ('confirmed', 'Tasdiqlangan'),
        ('cancelled', 'Bekor qilingan'),
        ('completed', 'Yakunlangan'),
    )
    user = models.ForeignKey(User, on_delete=models.CASCADE,
                             related_name='reservations')
    business = models.ForeignKey(Business, on_delete=models.CASCADE,
                                 related_name='reservations')
    room = models.ForeignKey(Room, on_delete=models.CASCADE,
                             related_name='reservations')
    availability = models.OneToOneField(Availability, on_delete=models.CASCADE,
                                       related_name='reservation')
    guests_count = models.PositiveIntegerField()
    special_request = models.TextField(blank=True)
    status = models.CharField(max_length=15, choices=STATUS_CHOICES,
                              default='pending')
    created_at = models.DateTimeField(auto_now_add=True)

```

7.5. reviews

```
# reviews/models.py
from django.db import models
from accounts.models import User
from businesses.models import Business
from reservations.models import Reservation

class Review(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
                             related_name='reviews')
    business = models.ForeignKey(Business, on_delete=models.CASCADE,
                                 related_name='reviews')
    reservation = models.OneToOneField(Reservation, on_delete=models.CASCADE,
                                       related_name='review')
    rating = models.PositiveSmallIntegerField() # 1-5
    comment = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

class ReviewPhoto(models.Model):
    review = models.ForeignKey(Review, on_delete=models.CASCADE,
                              related_name='photos')
    image = models.ImageField(upload_to='review_photos/')
```

7.6. subscriptions

```
# subscriptions/models.py
from django.db import models
from businesses.models import Business
from accounts.models import User

class SubscriptionPlan(models.Model):
    business_type = models.CharField(max_length=15,
                                     choices=Business.TYPE_CHOICES)
    monthly_price = models.DecimalField(max_digits=12, decimal_places=2)
    trial_days = models.PositiveIntegerField(default=7)

class Subscription(models.Model):
    STATUS_CHOICES = (
        ('trial', 'Trial (bepul)'),
        ('active', 'Faol'),
        ('expired', 'Muddati tugagan'),
    )
    business = models.OneToOneField(Business, on_delete=models.CASCADE,
                                   related_name='subscription')
    plan = models.ForeignKey(SubscriptionPlan, on_delete=models.PROTECT)
    status = models.CharField(max_length=10, choices=STATUS_CHOICES,
                             default='trial')
    trial_ends_at = models.DateTimeField()
    subscription_ends_at = models.DateTimeField(null=True, blank=True)
    approved_by = models.ForeignKey(User, null=True, blank=True,
                                    on_delete=models.SET_NULL)

class PaymentLog(models.Model):
    """To'lov Telegram orqali qo'lda amalga oshgani uchun,
    bu - admin qo'lda kiritadigan tarix yozuvi."""
    subscription = models.ForeignKey(Subscription, on_delete=models.CASCADE,
                                    related_name='payments')
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    confirmed_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True)
    note = models.CharField(max_length=255, blank=True)
    paid_at = models.DateTimeField(auto_now_add=True)
```

7.7. Ilovalar orasidagi bog'lanish sxemasi

```
accounts (User, BusinessApplication)
  |
  v
businesses (Business <- owner, application) --- (Room)
  |           |
  v           v
catalog (MenuItem, Package)  reservations (Availability, Reservation)
                               |
                               v
                               reviews (Review, ReviewPhoto)

businesses.Business --> subscriptions (Subscription, PaymentLog, SubscriptionPlan)
```

8. Asosiy API endpointlari (qisqacha ro'yxat)

Endpoint	Metod	Tavsif
/api/auth/register/	POST	Ro'yxatdan o'tish (full_name, email, phone, username, password)
/api/auth/verify-phone/	POST	SMS-kodni tasdiqlash
/api/auth/login/	POST	Kirish, JWT token olish
/api/business-application/	POST	'Restoran/To'yxona ochish' arizasi yuborish
/api/business-application//approve/	POST	(faqat super-admin) to'lovni tasdiqlash
/api/businesses/	GET	Qidiruv va filtr bilan ro'yxat (nom, masofa, sana/vaqt)
/api/businesses//	GET	Bitta biznesning to'liq profili (xona, menyu/paket, sharhlar)
/api/businesses//rooms/	GET	Biznesning xona/zallari
/api/businesses//availability/	GET	Berilgan sanadagi bo'sh vaqtlar
/api/reservations/	POST	Yangi bron yaratish
/api/reservations/my/	GET	Foydalanuvchining bron tarixi
/api/reservations//cancel/	POST	Bronni bekor qilish
/api/reviews/	POST	Sharh qoldirish (faqat yakunlangan bron uchun)

9. Funksional bo'lmagan (non-functional) talablar

- Barcha parollar bcrypt/Django default hasher orqali saqlanadi.
- API JWT orqali himoyalangan, business rolidagi foydalanuvchi faqat o'z Business'iga tegishli ma'lumotlarni tahrirlay oladi (permission class orqali tekshiriladi).
- Availability va Reservation yaratishda database-level tranzaksiya (select_for_update) ishlatilishi shart — ikki mijoz bir vaqtni bir vaqtda band qilib qo'ymasligi uchun.
- Trial va obuna muddatlarini kuzatish uchun Celery + Celery Beat orqali har kuni ishga tushadigan job (check_expired_subscriptions) bo'lishi kerak.
- Rasmlar uchun S3-mos ombor (masalan MinIO) ishlatiladi, lokal diskda saqlanmaydi (production uchun).

10. Ochiq savollar (jamoalar bilan kelishilishi kerak)

- Restoran uchun aniq oylik obuna narxi qancha bo'ladi? (to'yxona uchun 300 000 so'm belgilangan)
- Trial muddati tugagach, profil butunlay yashiriladimi yoki 'cheklangan' holatda ko'rinishda qoladimi?
- Telegram bot orqali to'lovni avtomatlashtirish keyingi bosqichda rejalashtirilganmi (masalan Click/Payme Telegram bot integratsiyasi)?
- Bitta foydalanuvchi bir nechta biznes (masalan ham restoran, ham to'yxona) ocha oladimi?

Ushbu hujjat UVENT jamoasi uchun tayyorlangan texnik topshiriq (TZ) bo'lib, ishlab chiqish jarayonida jamoa tomonidan aniqlashtirilishi va kengaytirilishi mumkin.